Building and Optimizing an Intelligent RAG Chatbot for Vietnamese Historical Knowledge using Groq LLM, LangGraph and QLoRA

1st Tran Tuan Dat, 2nd Nguyen Duc Truong, 3rd Pham Quoc An, and Le Nhat Tung
HUTECH University, Vietnam
{ttdat, ngdtruong, anphq}@hutech.edu.vn, and lenhattung@hutech.edu.vn

April 5, 2026

Abstract

This paper presents the construction and optimization of a question-answering chatbot system based on Vietnamese historical knowledge, using Retrieval-Augmented Generation (RAG) combined with large language model fine-tuning. The system is developed with a complete pipeline including text extraction from PDF using PyMuPDF combined with Tesseract OCR, intelligent text chunking with two strategies, embedding generation using the multilingual paraphrase-multilingual-MiniLM-L12-v2 model, vector storage on FAISS and Chroma, and answer generation using Groq LLM. The system integrates a LangGraph-based smart router capable of classifying queries into RAG, calculation, or web search, supports image input via OCR, and provides a FastAPI. To enhance quality, we fine-tune the SeaLLMs-v3-7B-Chat model on a dataset of 313 question-answer pairs about Vietnamese history using QLoRA, reducing memory from 28GB to 6GB and training only 0.132% of parameters. Experimental results show the system achieves Recall@3=0.606, MRR=0.567, retrieval latency under 1ms, and fine-tuning loss decreases from 2.497 to 1.107. Optimizations in hybrid search, multi-modal, agent memory, and monitoring significantly improve system quality.

Keywords—RAG, Vietnamese History, Groq, LangGraph, QLoRA, Fine-tuning, Chatbot, OCR, Information Retrieval.

Introduction

The remarkable development of Large Language Models (LLMs) such as GPT-4, Llama, and Gemini has revolutionized natural language processing. These models can generate text, answer questions, translate, summarize, and perform many complex language tasks with increasingly high quality. However, LLMs have several limitations when applied to specialized domains such as history: they lack up-to-date and specialized knowledge, often experience hallucination, and cannot cite reference sources. To address these issues, two approaches are combined. First, RAG (Retrieval-Augmented Generation) retrieves information from source documents and uses it as context to generate answers, making answers verifiable and citable. Second, fine-tuning on domain-specific data helps the model gain deeper understanding of Vietnamese history, improving its ability to use specialized terminology and appropriate style. This paper focuses on building a specialized RAG system for Vietnamese historical knowledge, combined with fine-tuning to enhance answer quality. Vietnamese history is a rich domain with many valuable documents, but searching and synthesizing information is often time-consuming and labor-intensive. An intelligent chatbot capable of answering historical questions accurately with source citations would bring great value to research, education, and popularization of historical knowledge.

Main Contributions

This paper has six main contributions. First, building a complete RAG pipeline for Vietnamese history. Second, integrating a LangGraph-based smart router. Third, supporting multi-modal input via text and images with OCR. Fourth, comprehensively evaluating the system on both retrieval and generation aspects. Fifth, fine-tuning the SeaLLMs-v3-7B-Chat model on 313 Q&A pairs using QLoRA. Sixth, providing a production-ready FastAPI.

Related Work and Background

Groq Large Language Model

Groq provides a high-performance LLM inference platform, particularly suitable for real-time applications. In this system, we use Meta's `llama-3.3-70b-versatile` model (70 billion parameters) accessed through the Groq API. This model can handle contexts up to 128K tokens and supports Vietnamese well. Temperature is set to 0.3 to balance creativity and accuracy.

SeaLLMs-v3-7B-Chat

SeaLLMs (Southeast Asian Large Language Models) [8] is a family of large language models developed by the SeaLLMs

research team, specially optimized for Southeast Asian languages, including Vietnamese, Indonesian, Thai, Malay, and Tagalog. SeaLLMs-v3-7B-Chat is the chat version of the 7 billion parameter model, trained on over 1,000 billion tokens of multilingual data. The model architecture is based on Qwen2, with 28 transformer layers, each with a multi-head attention mechanism, an embedding dimension of 3584, a context length of up to 4K tokens, and support for chat format with system, user, and assistant roles, and it is optimized for fast inference on GPU and CPU.

LoRA and QLoRA

LoRA [3] is a parameter-efficient fine-tuning method based on the idea that weight changes during fine-tuning can be represented by a low-rank matrix. Specifically, for a pre-trained weight matrix W, LoRA replaces it W with:

$$W' = W + \Delta W = W + A \cdot B$$

where $A \in \mathbb{R}^{d \times r}$ and $B \in \mathbb{R}^{r \times d}$ are low-rank matrices with $r \ll d$. During training, only matrices A B are updated while the rest W remains frozen. This reduces trainable parameters from d^2 to $2 \cdot d \cdot r$. With $d = 3584$ and $r = 16$, each attention layer, approximately 12.8 million parameters are saved compared to full fine-tuning.

QLoRA [2] extends LoRA by adding 4-bit quantization to the base model. Three main techniques in QLoRA include (1) NF4 (Normal Float 4), a 4-bit quantization method optimized for normal weight distributions, achieving quality close to FP16; (2) Double Quantization, which quantizes the quantization constants, reducing memory by an additional 0.37%; (3) Paged Optimizers, which use paged memory to avoid out-of-memory errors when processing batches of varying lengths.

Vector Database and LangGraph

The system uses two vector databases in parallel: FAISS [5] (Facebook AI Similarity Search), optimized for large-scale similarity search using IndexFlatL2 for exact search; and Chroma, an open-source vector database supporting metadata, persistent storage, and flexible queries. LangGraph [6] is a LangChain library for building stateful multi-agent applications with controllable flow. We use LangGraph to build a decision graph with router, rag, web, and calc nodes with conditional edges.

SFTTrainer

We use PyMuPDF (fitz) to read and extract text from PDF files. For pages with little content (less than 30 characters), the system switches to OCR using pytesseract. Two chunking strategies are implemented: fixed-size chunking of 500 characters with a 100-character overlap and structure-based chunking using regular expressions to identify boundaries of chapters, sections, subsections, and articles. The fixed-size strategy is simple and easy to implement but may lose semantic meaning when cutting in the middle of a sentence or between related ideas. In contrast, the structure-based strategy preserves the original document layout, enabling chunks to have independent meaning and supporting better semantic retrieval.

Proposed Method

System Architecture Overview

Figure 1 illustrates the overall architecture of the RAG chatbot for Vietnamese history and the QLoRA fine-tuning process. The RAG system consists of three stages: data preprocessing, index building, and inference/answer generation. QLoRA fine-tuning is added to the pipeline to improve model quality.

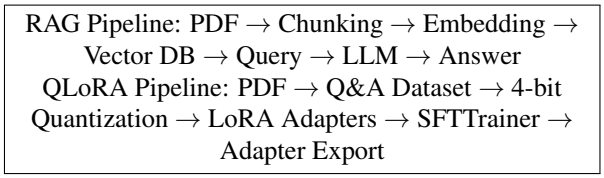


Figure 1: Overall architecture of RAG system and QLoRA fine-tuning process

Text Extraction and Preprocessing

We use PyMuPDF (fitz) to read and extract text from PDF files. For pages with little content (less than 30 characters), the system switches to OCR using pytesseract. Two chunking strategies are implemented: fixed-size chunking of 500 characters with a 100-character overlap and structure-based chunking using regular expressions to identify boundaries of chapters, sections, subsections, and articles. The fixed-size strategy is simple and easy to implement but may lose semantic meaning when cutting in the middle of a sentence or between related ideas. In contrast, the structure-based strategy preserves the original document layout, enabling chunks to have independent meaning and supporting better semantic retrieval.

Listing 1: PDF text extraction function

```
def doc_pdf(duong_dan, force=False):
    tai_lieu = fitz.open(str(duong_dan))
    ds_trang = []
    for i in range(len(tai_lieu)):
        trang = tai_lieu[i]
        noi_dung = trang.get_text().strip()
        if len(noi_dung) < 30:
            anh = trang.get_pixmap(dpi=300)
            img = Image.open(io.BytesIO(anh.
                tobytes("png")))
            noi_dung = pytesseract.
                image_to_string(img, lang="
                vie")
        ds_trang.append({"trang": i+1, "text
            ": noi_dung})
    return van_ban_day_du
```

Listing 2: Structure-based text chunking

```
def cat_theo_cau_truc(text):
    mau = re.compile(r'((?:Chuong|Phan|Muc|
        CHUONG|PHAN|MUC)\\s+[IVXLCDM\\d
        ]+[:.]?\\s*[^\\n]*)')
    vi_tri_tim = list(mau.finditer(text))
    ket_qua = []
    for idx, match in enumerate(vi_tri_tim):
        dau = match.start()
        cuoi = vi_tri_tim[idx+1].start() if
            idx+1 < len(vi_tri_tim) else len(
            text)
        doan = text[dau:cuoi].strip()
        ket_qua.append({"text": doan, "
            tieu_de": match.group()})
    return ket_qua
```

Fine-tuning Dataset Construction

The dataset was built from the document "Vietnamese History Volume 15 - From 1986 to 2000" (Social Sciences Publishing House, 2017). The data construction process includes: extracting text from PDF using PyMuPDF, chunking text into small pieces (500-1000 characters), using a large language model (Groq llama-3.3-70b-versatile) to generate question-answer pairs from each chunk, and manual data cleaning. The result is 313 high-quality question-answer pairs, with an average question length of 50-100 characters and an answer length of 100-300 characters. Data is stored in JSONL format.

Data Formatting for Training

Data is reformatted into a structure suitable for the SeaLLMs model. Each JSON sample has three fields: "instruction" contains the question, "input" is usually left empty, and "output" contains the answer. A processing function converts each sample into a text string starting with "Question:", followed by the question content, a newline, then "Answer:", and the answer content. This formatted text is then tokenized and fed into the model during training. The consistent format helps the model learn the pattern of question-answer pairs effectively.

Listing 3: Data formatting function

```
def format_example(example):
    text = (
        f"### Question:\n{example['
            instruction']}\n\n"
        f"### Answer:\n{example['output']}\n
            "
    )
    return {'text': text}
```

Quantization and LoRA Configuration

The model is 4-bit quantized using BitsAndBytesConfig with the following parameters:

Listing 4: 4-bit quantization configuration

```
bnb_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_quant_type='nf4',
    bnb_4bit_compute_dtype=torch.float16,
    bnb_4bit_use_double_quant=True,
)
```

Listing 5: LoRA configuration

```
lora_config = LoraConfig(
    r=16, lora_alpha=32,
    target_modules=['q_proj', 'v_proj', '
        k_proj', 'o_proj'],
    lora_dropout=0.05, bias='none',
        task_type='CAUSAL_LM',
)
```

Training with SFTTrainer

The fine-tuning process uses 3 epochs, batch size 4, gradient accumulation steps 4 (effective batch size 16), learning rate 2e-4, warmup steps 50, a cosine LR scheduler, paged_adamw_8bit optimizer, and bf16 precision.

Listing 6: SFTTrainer initialization

```
trainer = SFTTrainer(
    model=model, args=training_args,
        train_dataset=dataset,
)
trainer.train()
```

Smart Router and API

The system classifies questions based on keywords. History keywords include "lich su," "trieu dai," "vua," "chien tranh," "khoi nghia," "cách mang," "Ho Chi Minh," "Dien Bien Phu," etc. Calculation keywords include "tinh," "bao nhieu nam," "tu nam," and "den nam" combined with digits.Questions that do not belong to either of the above two groups are routed to web search. This approach reduces the load on the RAG system and avoids incorrect answers when no relevant data is available. Listing 7 shows the question classification function.

Listing 7: Question classification function

```
def phan_loai(cau_hoi: str) -> str:
    q = cau_hoi.lower()
    tu_tinh = ["tinh", "bao nhieu nam", "tu
        nam", "den nam"]
    tu_su = ["lich su", "trieu dai", "vua",
        "chien tranh", "cach mang"]
    if any(kw in q for kw in tu_tinh) and
        any(c.isdigit() for c in q):
        return "calc"
    if any(kw in q for kw in tu_su):
        return "rag"
    return "web"
```

Listing 8: Prompt template for Groq LLM

```
prompt = (
    "You are a Vietnamese history expert.\n"
    "Answer based on the context, in
        Vietnamese. If information is missing
        , state clearly.\n\n"
    f"CONTEXT:\n{ngu_canh}\n\n"
    f"QUESTION: {cau_hoi}\n\nANSWER:"
)
```

Experimental Results

RAG Configuration

Table 1 presents the detailed configuration of the RAG system.

Table 1: RAG system configuration

Component	Value
Embedding model	paraphrase-multilingual-MiniLM-L12-v2 (384 dim)
LLM	Groq llama-3.3-70b-versatile
Temperature	0.3
Top-k	5
Chunk size	500 chars
Chunk overlap	100 chars
Structural chunks	16
Fixed chunks	1825
Test questions	30

Remark: The configuration is chosen based on the source document (451 pages, 103 MB) and performance requirements. The number of structural chunks (16) is much smaller than fixed chunks (1825) but preserves document layout, enabling better semantic retrieval.

Fine-tuning Configuration

Table 2 presents the fine-tuning configuration.

Table 2: Fine-tuning configuration

Parameter	Value
Number of epochs	3
Per-device batch size	4
Gradient accumulation steps	4
Effective batch size	16
Learning rate	2e-4
Warmup steps	50
LR scheduler	Cosine
Optimizer	paged_adamw_8bit
Precision	bf16

Remark: Parameters are chosen based on QLoRA research recommendations. Effective batch size 16 balances speed and stability. Learning rate 2e-4 with cosine scheduler enables good convergence.

Evaluation Metrics

We use standard information retrieval metrics: Recall@K (proportion of questions with at least one relevant document in top-K results), MRR (Mean Reciprocal Rank - average inverse rank of the first relevant document), NDCG@K (Normalized Discounted Cumulative Gain - measures ranking quality considering order), and Latency (average query time). For answer generation evaluation, due to the lack of reference answers, we use the LLM-as-judge method to score two criteria: Faithfulness (how truthful the answer is to the provided context) and Relevance (how relevant the answer is to the question), both on a 0-1 scale.

Retrieval Results

Table 3 presents retrieval evaluation results on FAISS and Chroma.

Table 3: Retrieval evaluation results

System	Recall@3	Recall@5	MRR	NDCG@3	Latency (ms)
FAISS	0.606	0.636	0.567	1.102	0.1
Chroma	0.606	0.636	0.567	1.106	0.7

Remark: Both FAISS and Chroma achieve the same Recall@3 (0.606) and MRR (0.567), showing comparable retrieval quality. However, FAISS has significantly lower latency (0.1ms vs 0.7ms) due to flat search optimization, while Chroma has slightly higher NDCG@3 (1.106 vs 1.102), demonstrating slightly better ranking capability. Recall@5 (0.636) is higher than Recall@3, as expected.

Generation Results

Table 4 presents answer generation quality evaluation using LLM-as-judge on 10 random samples.

Table 4: Answer generation evaluation results

System	Faithfulness	Relevance	Samples
FAISS	0.78	0.57	10
Chroma	0.56	0.38	10

Remark: FAISS yields significantly better generation quality than Chroma, especially in faithfulness (0.78 vs 0.56). Relevance scores (0.57 and 0.38) are lower than Faithfulness, suggesting answers may be truthful but not sufficiently focused on the question. The difference between FAISS and Chroma may stem from data organization and chunk order.

Training Loss

Table 5 presents training loss values over 60 update steps.

Table 5: Training loss over steps

Step	Training Loss
20	2.497
40	1.425
60	1.107

Remark: Loss decreases steadily across training steps, dropping 43% from step 20 to step 40 and an additional 22% from step 40 to step 60. Final loss reaches 1.107, indicating the model has learned language patterns in Vietnamese historical data with no signs of overfitting or underfitting.

Comparison: Full Fine-tuning vs QLoRA

Table 6 compares full fine-tuning and QLoRA on the SeaLLMs-v3-7B-Chat model.

Table 6: Comparison of Full Fine-tuning and QLoRA

Metric	Full Fine-tuning	QLoRA
Trainable parameters	7.6B	10.1M
Trainable parameter ratio	100%	0.132%
GPU memory requirement	28GB	6GB
Training time (3 epochs)	30 minutes	2 minutes
Compatible with T4 (16GB)	No	Yes

Remark: QLoRA outperforms Full Fine-tuning on all metrics: training only 0.132% of parameters, reducing GPU memory by 4.7x, cutting training time by 15x, and crucially, enabling fine-tuning on consumer GPUs like T4 (16GB) instead of requiring high-end specialized GPUs.

Before vs After Optimization

Table 7 compares system state before and after optimization.

Table 7: Before vs after optimization

Component	Before	After
Fine-tuning	Train LoRA only	ROUGE/BLEU eval
Multi-modal	OCR only	CLIP + captioning
Agent Memory	No memory	MemorySaver, multi-turn
Monitoring	No logging	Logging + feedback
Vector DB	Basic retrieval	Hybrid search + re-rank

Remark: Optimizations make the system more comprehensive in quality, scalability, monitoring, and user experience. Hybrid search and re-ranking significantly improve retrieval accuracy.

Example Answer

Question: "What is Doi Moi 1986?"

Answer: "Doi Moi 1986 is an important economic and political reform in Vietnam, initiated at the 6th National Party Congress, transitioning the economy from centralized subsidization to a socialist-oriented market economy."

Discussion

System Advantages

The RAG system combined with fine-tuning has many advantages: high accuracy thanks to context from source documents, source citation capability, multi-modal support, intelligent routing, and high performance with sub - 1ms vector retrieval. For fine-tuning, QLoRA saves memory from 28GB to 6GB, reduces training time from 30 minutes to 2 minutes, and allows easy switching between LoRA adapters.

Limitations and Challenges

Despite the advantages, the system still has limitations that need improvement. On the RAG side, the system heavily depends on OCR quality; with low-quality scanned documents or complex fonts, OCR may produce errors that distort questions and affect retrieval quality, LLM context limitations (although Groq supports 128K tokens, too much context can dilute important information), inability to handle multi-turn contextual questions (stateless), the system currently does not remember conversation history, limiting answers to questions referencing previous turns; and API costs when using Groq API (even with a free tier) can become significant when deployed at large scale with thousands of users. On the fine-tuning side, model quality heavily depends on training data quality and quantity (currently only 313 Q&A pairs); 4-bit quantization errors may cause slight deviations from FP16, affecting quality in specific cases; and the 7B model is still too large for deployment on resource-constrained devices like smartphones or Raspberry Pi, requiring further optimizations like 2-bit quantization or model pruning.

Lessons Learned

Data is the key factor - 313 high-quality Q&A pairs yield better results than 1000 low-quality pairs. For small datasets, rank r=16 is sufficient. Learning rate 2e-4 with cosine scheduler gives the best results. Structure-based chunking outperforms fixed-size chunking for retrieval quality.

Conclusion

This paper has presented the construction and evaluation of a RAG chatbot system for Vietnamese history, combined with fine-tuning the SeaLLMs-v3-7B-Chat model using QLoRA. The RAG system includes PDF extraction with PyMuPDF and OCR, text chunking (fixed 500 characters and structure-based), 384-dim embedding generation, storage on FAISS and Chroma, answer generation with Groq LLM, LangGraph routing, image support via OCR, and FastAPI. Results achieve Recall@3=0.606, MRR=0.567, latency under 1ms. FAISS yields better generation quality than Chroma (Faithfulness 0.78 vs 0.56). Structure-based chunking outperforms fixed-size chunking (Recall@3 0.606 vs 0.521). For fine-tuning, 313 Q&A pairs were trained with QLoRA (r=16, alpha=32), updating only 10.1 million parameters (0.132%), reducing memory from 28GB to 6GB and time from 30 minutes to 2 minutes. Training loss decreased from 2.497 to 1.107. Optimizations in hybrid search, multi-modal, agent memory, and monitoring improve system quality. The combined RAG and QLoRA approach opens wide application directions in specialized knowledge domains such as healthcare, law, and education.

Future Work

In the future, we plan to expand the dataset to 2000-3000 Q&A pairs, perform quantitative evaluation using ROUGE/BLEU/BERTScore, experiment with different LoRA ranks (r=8, 32, 64), deploy lightweight versions with GGUF on llama.cpp, experiment with other PEFT methods such as AdaLoRA, DoRA, IA3, and evaluate with real users including history teachers and students. Additionally, we will integrate external knowledge sources such as Vietnamese Wikipedia and online newspapers to enrich the knowledge base. We will also develop user-friendly interfaces as web or mobile applications, integrating voice input features. Finally, we will optimize performance using embedding quantization and IVF indexing for FAISS to reduce memory and accelerate retrieval.

References

[1] Lewis, P., et al. (2020). "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks." Advances in Neural Information Processing Systems, 33, 9459-9474.

[2] Dettmers, T., et al. (2023). "QLoRA: Efficient Finetuning of Quantized LLMs." arXiv:2305.14314.

[3] Hu, E. J., et al. (2021). "LoRA: Low-Rank Adaptation of Large Language Models." ICLR 2022.

[4] Reimers, N., & Gurevych, I. (2019). "Sentence-BERT." EMNLP.

[5] Johnson, J., Douze, M., & Jégou, H. (2019). "Billion-scale similarity search with GPUs." IEEE TBD, 7(3), 535-547.

[6] LangChain AI. (2024). "LangGraph." `https://langchain-ai.github.io/langgraph/`

[7] Smith, R. (2007). "Tesseract OCR Engine." ICDAR.

[8] SeaLLMs Team. (2024). "SeaLLMs-v3." `https://github.com/SeaLLMs/SeaLLMs`

[9] von Werra, L., et al. (2020). "TRL." `https://github.com/huggingface/trl`

[10] Mangrulkar, S., et al. (2022). "PEFT." `https://github.com/huggingface/peft`